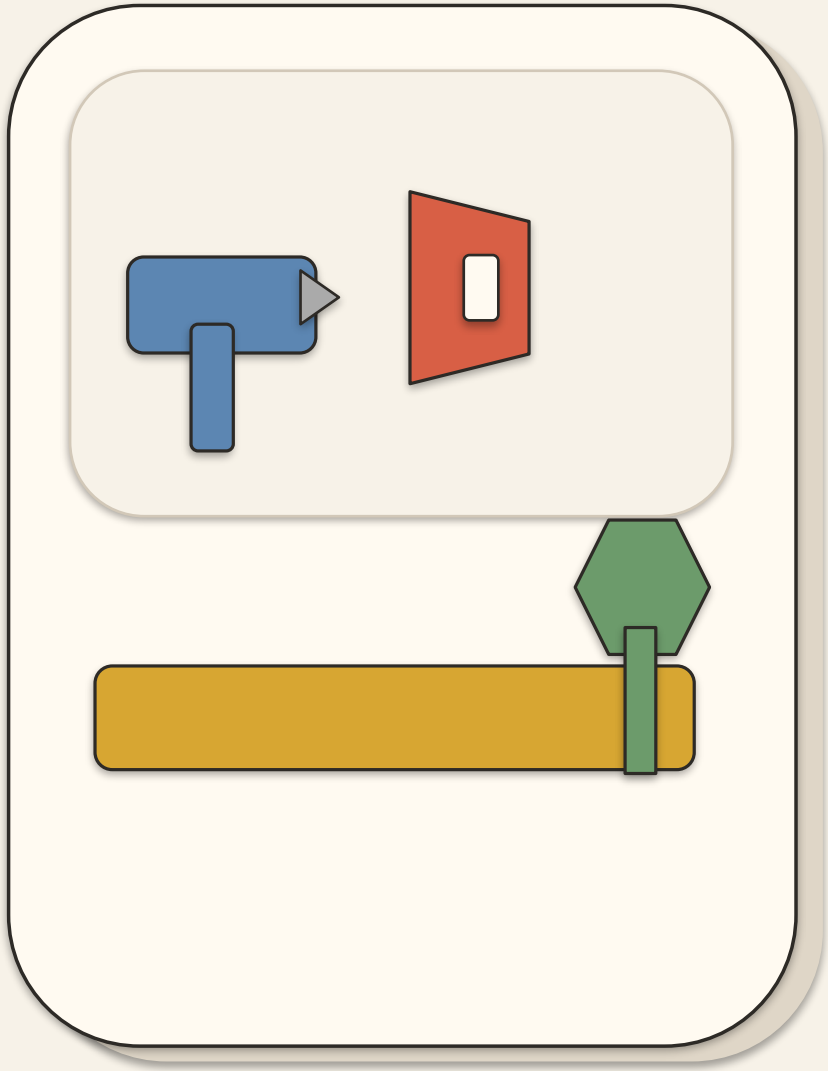
OPENING PERSPECTIVE

AI-assisted development can feel like walking into a workshop full of unfamiliar power tools.

Useful. Powerful. Fast-moving.
Also a little intimidating until you know what each tool does and where it belongs.

Today
shared vocabulary | practical workflows
live demo | open floor / Q&A



New tools arrive faster than most teams have time to fully research them.

WHY THIS MATTERS

Most confusion comes from calling all of this just “AI.”

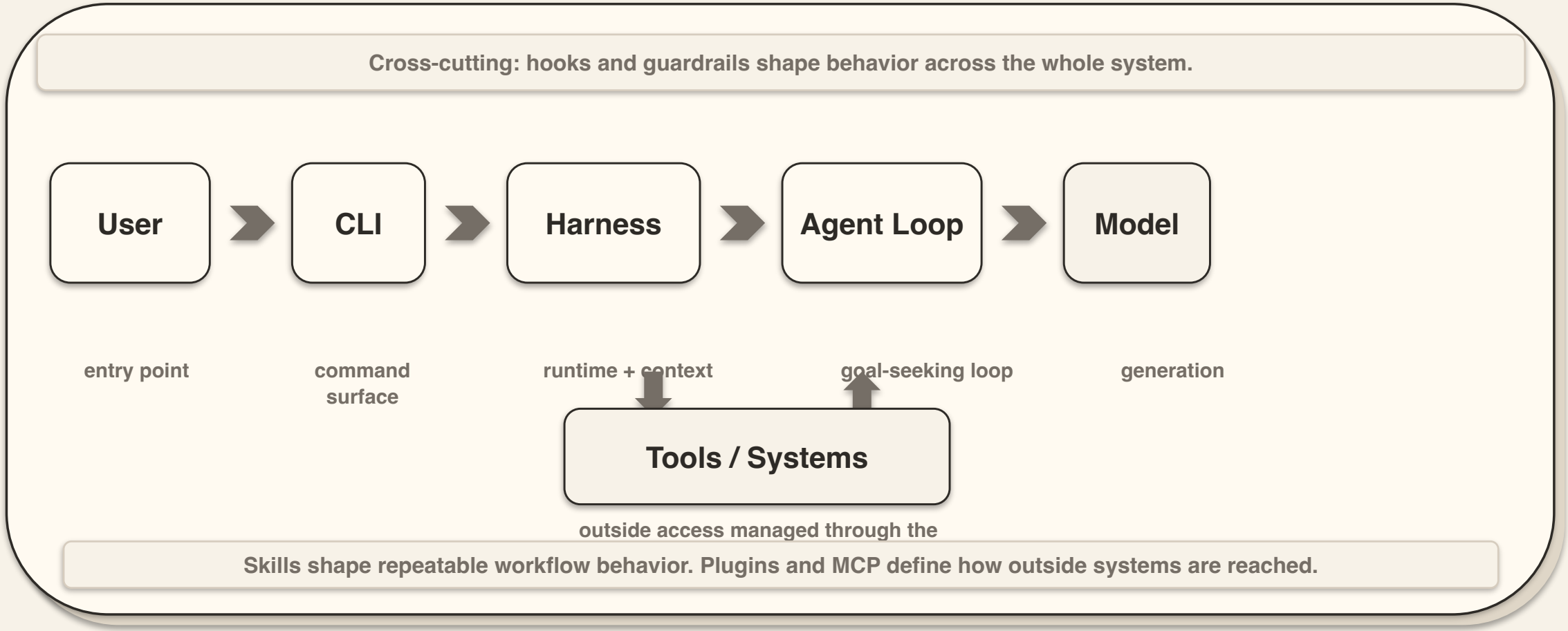
Once you separate the layers, the design, debugging, permissions, and safety story becomes much easier to reason about.

Today’s job is to separate the pieces before we judge the tools.

Think in layers, not one magic box.

Familiar concept	AI / agentic layer
entry point	**CLI**
runtime	**harness**
execution loop	**agent**
hard boundaries	**rules / guardrails**
reusable process	**skills**
outside access	**plugins / MCP**

A working agent system is a stack, not one magic model call.



MODEL

The model is the part that generates the next output from the input it sees.

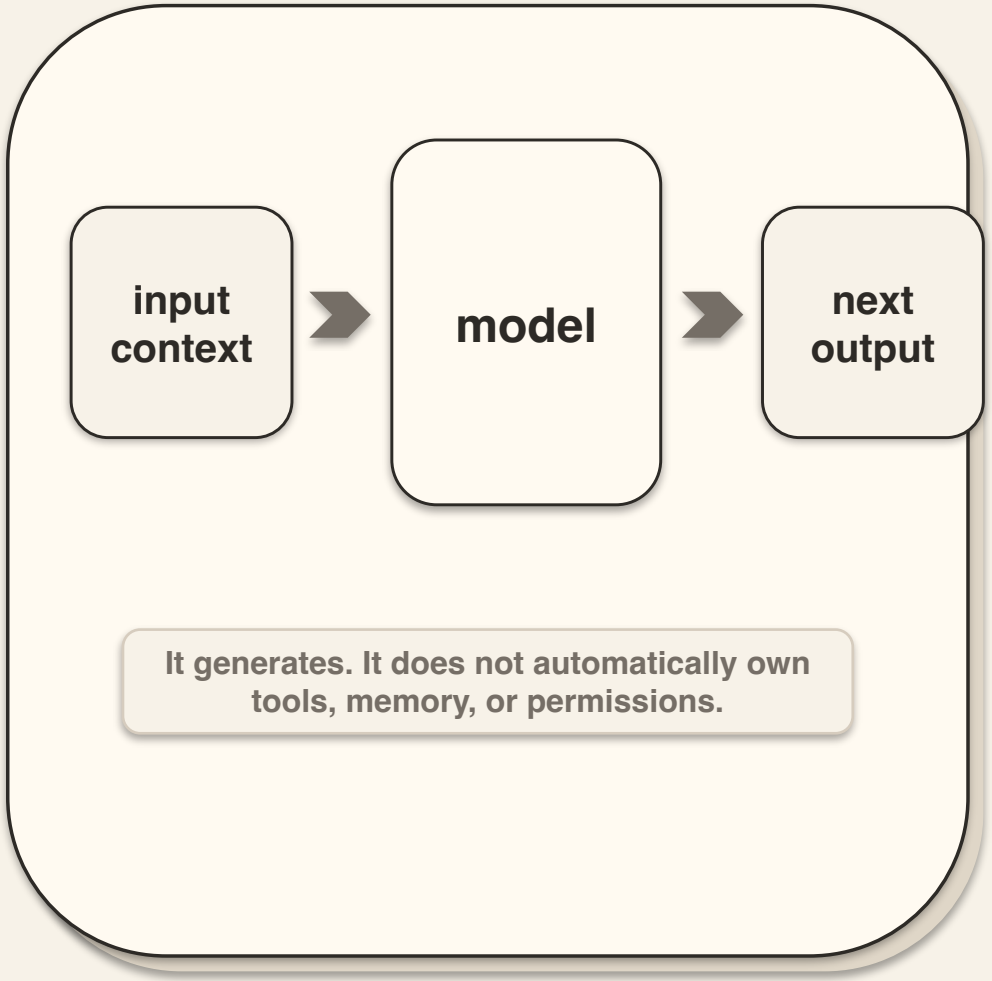
Powerful, but not the whole system.

Provider examples

OpenAI | Anthropic | Google

Model families

GPT | Claude | Gemini

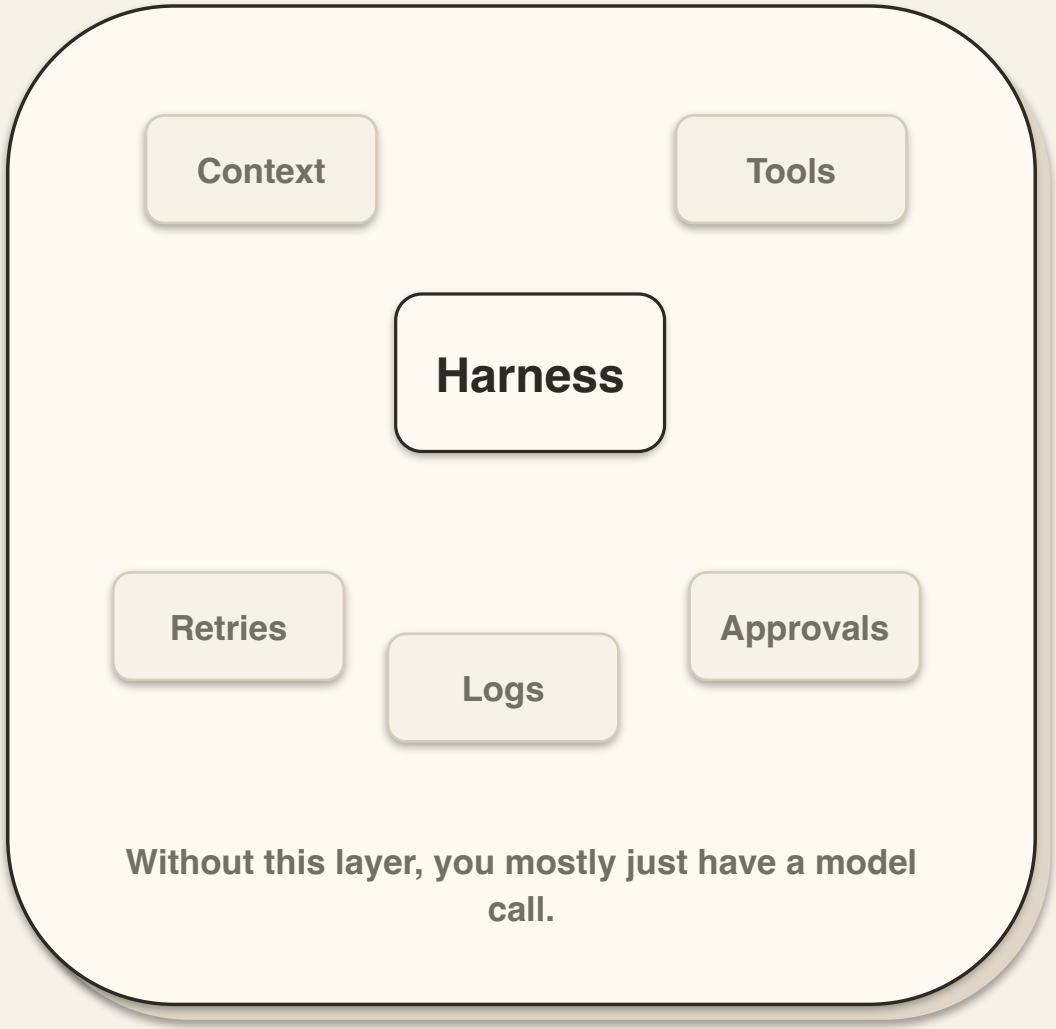


The harness is the runtime around the model.

It manages context, tools, retries, approvals, and execution flow.

Runtime examples

- Claude Code
- Copilot CLI
- Codex CLI



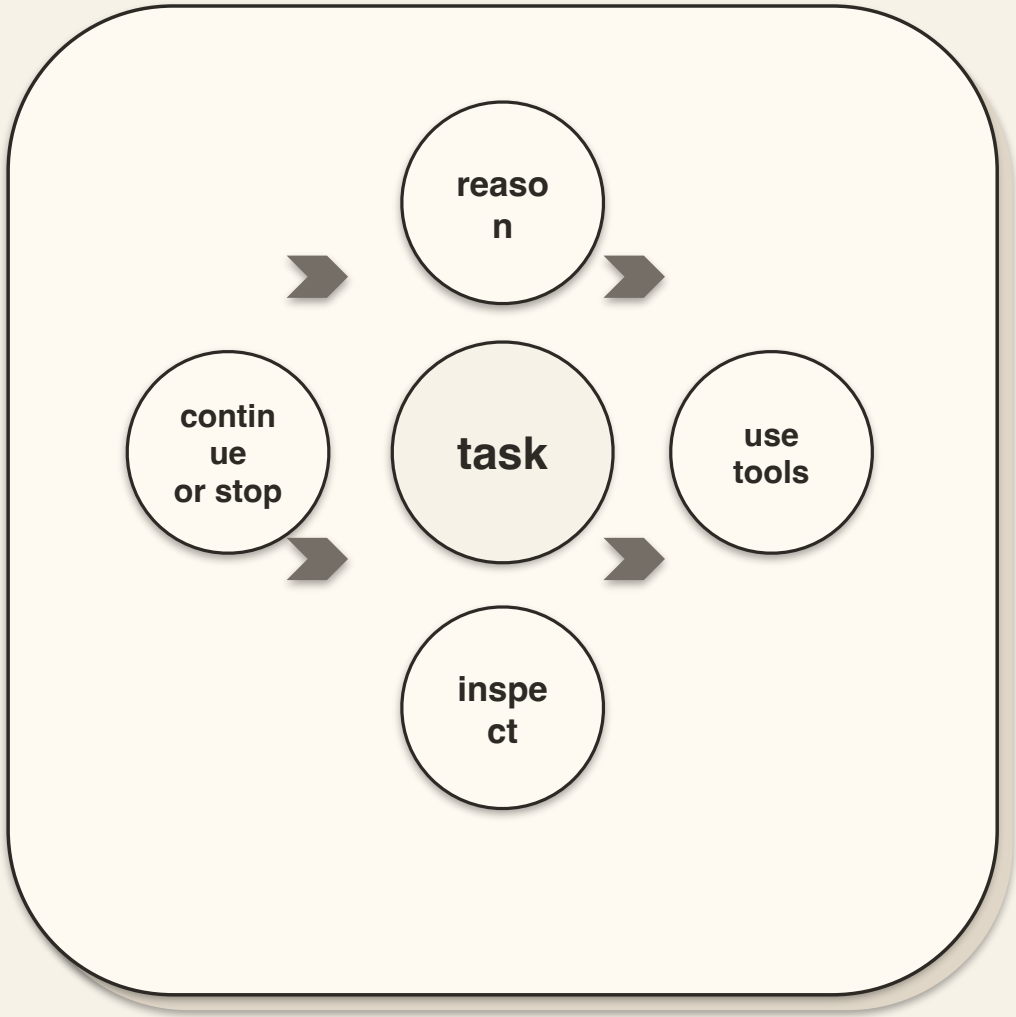
AGENT

An agent is the goal-seeking loop that keeps going until the task is done or must stop.

Think loop, not magic teammate.

Contrast

- single model call
- coding agent investigating a bug



Subagents are delegated workers for bounded, independent tasks.

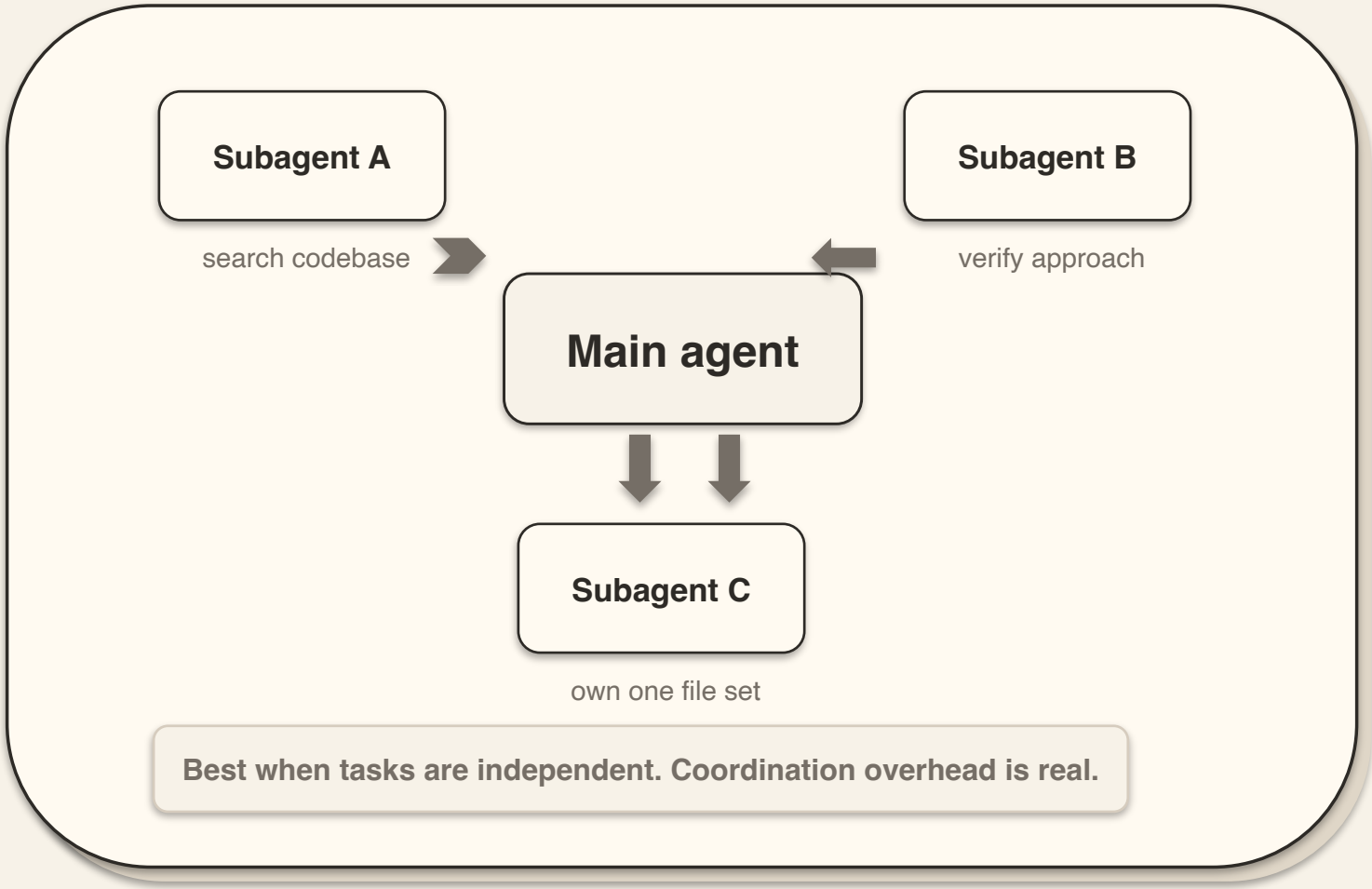
Useful for

- parallel research
- verification pass

Good fit

parallel work
clear ownership
non-blocking

Bad fit critical path work



Request, instructions, and rules are not the same thing.

Request

The current task for this run.
Think: the order ticket for this one meal.

Example

- current task

Instructions

Standing guidance loaded around every task.
Think: the station card pinned above the counter.

Example

- copilot-instructions
- project-context.md

Rules

Non-negotiable boundaries the system cannot violate.
Think: the health code or thermometer threshold.

Example

- must be safe
- approval required

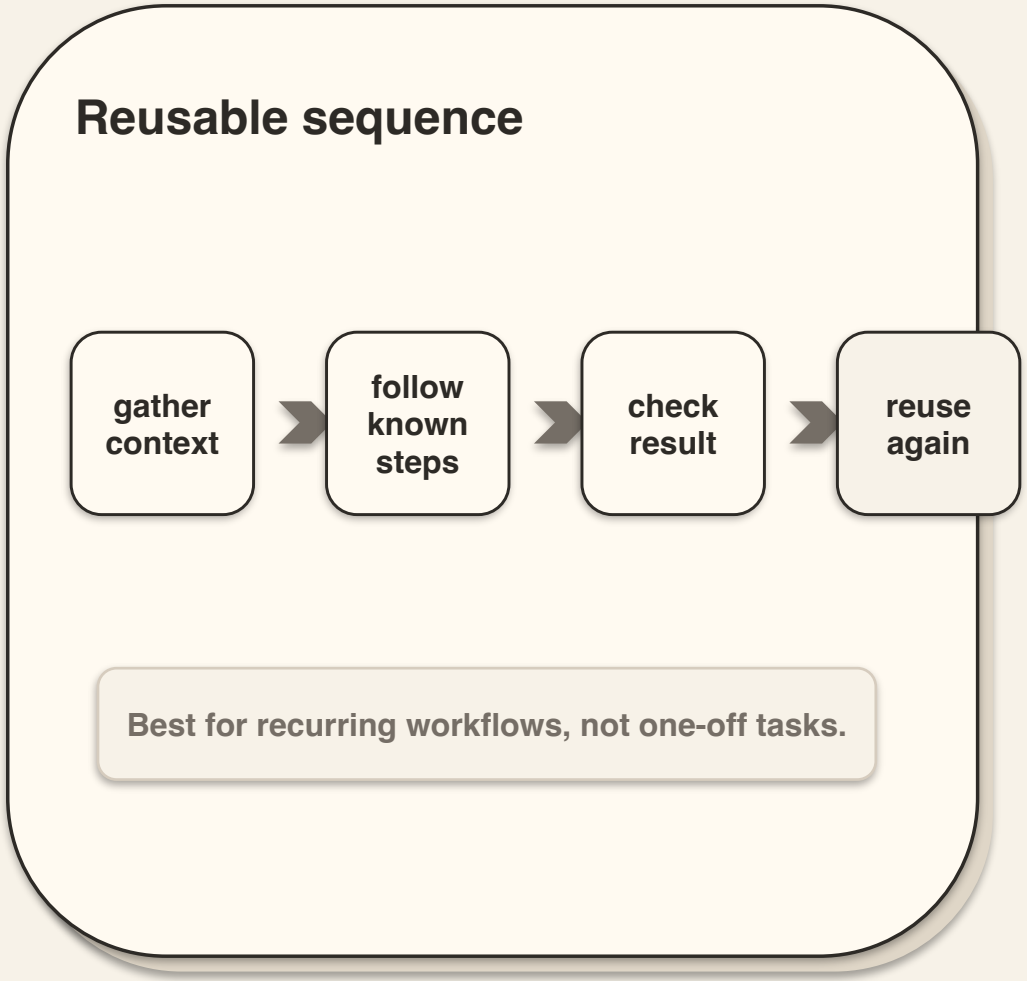
SKILL

A skill is a reusable way of doing a known task well.

It packages repeatable workflow logic so you do not have to re-describe the same process every time.

Examples

- debug a bug
- summarize a spec
- review a PR



Use rules for boundaries, instructions for steering, and skills for reuse.

Dimension	Instructions	Rules	Skills
Best for	steady guidance	hard boundaries	reusable workflows
If overused	bloated prompt	rigid behavior	skill sprawl
Reuse story	repeated every run	repeated every run	built once, reused when relevant

Keep instructions lean. Push repeated procedures into skills.

Examples

- copilot-instructions
- AGENTS.md
- PR review workflow

A plugin adds capability. MCP is the standard way systems connect.

Plugin

A plugin gives the system a new thing it can do.

Plugin examples

- GitHub
- Slack
- database

MCP

MCP gives different tools a common connection stand

MCP idea

- shared protocol
- same connection standard

The CLI is the command surface where the user tells the system what to do.

Important because developers care about repeatability and automation.

Why it matters

- run task
- inspect output
- repeat command

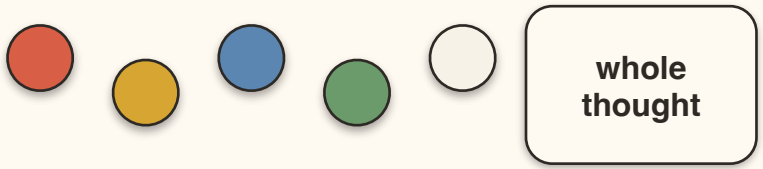
terminal / command surface

```
$ run review-task
> load context
> inspect files
> run checks
> summarize findings
```

A good CLI makes the workflow visible and repeatable.

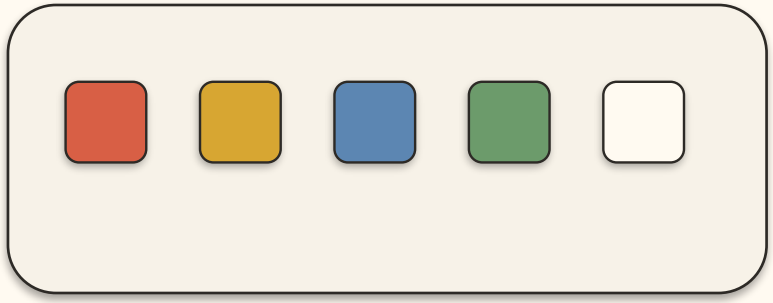
Tokens are the units. The context window is the active capacity.

Token



The model works on many small text pieces, not full ideas all at once.

Context window



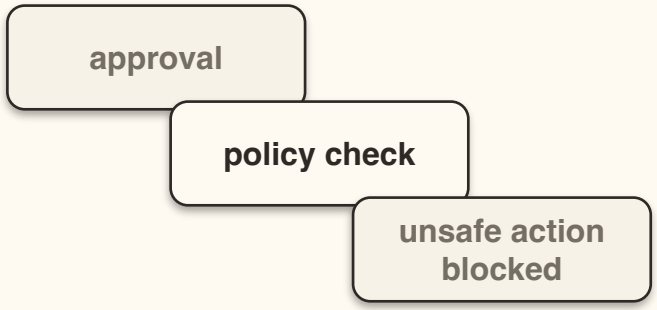
Only so much active information can stay in play before the workspace gets crowded.

Hooks are trigger points. Guardrails are layered checks around execution.

Hook

A hook is a defined point where extra logic can run.

Guardrails



Guardrails are layered checks that keep behavior inside safe bounds.

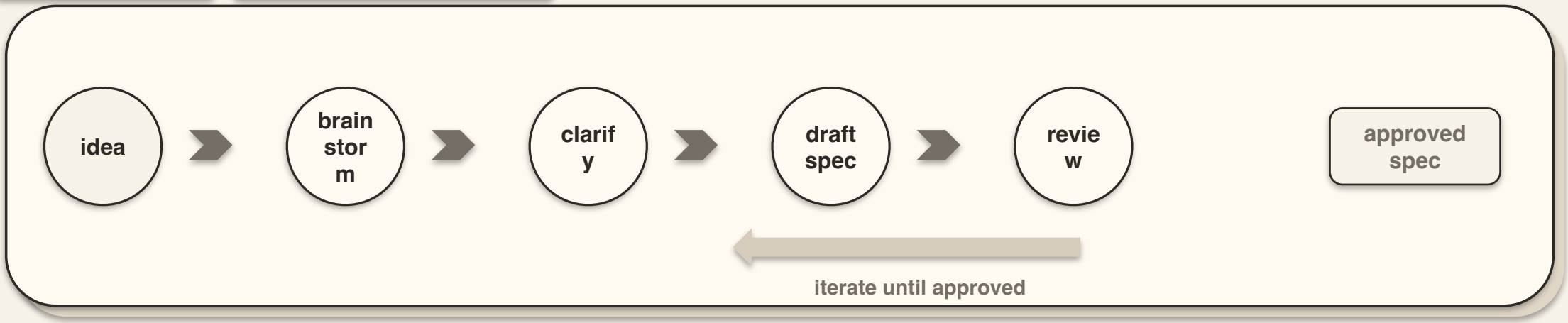
The first loop is not coding. It is refining the request until it becomes an approved spec.

This is where fuzzy requests become structured intent.

Workflow examples

BMAD: quick-spec

superpowers: brainstorming



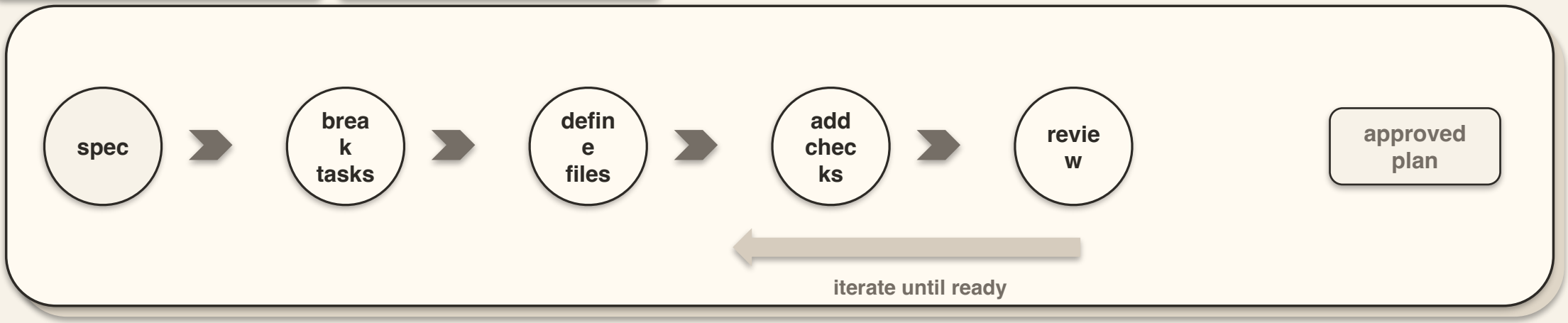
Once the spec is accepted, the next loop turns it into a concrete plan.

This is the difference between a good idea and a buildable plan.

Workflow examples

BMAD: architecture -> epics

superpowers: writing-plans



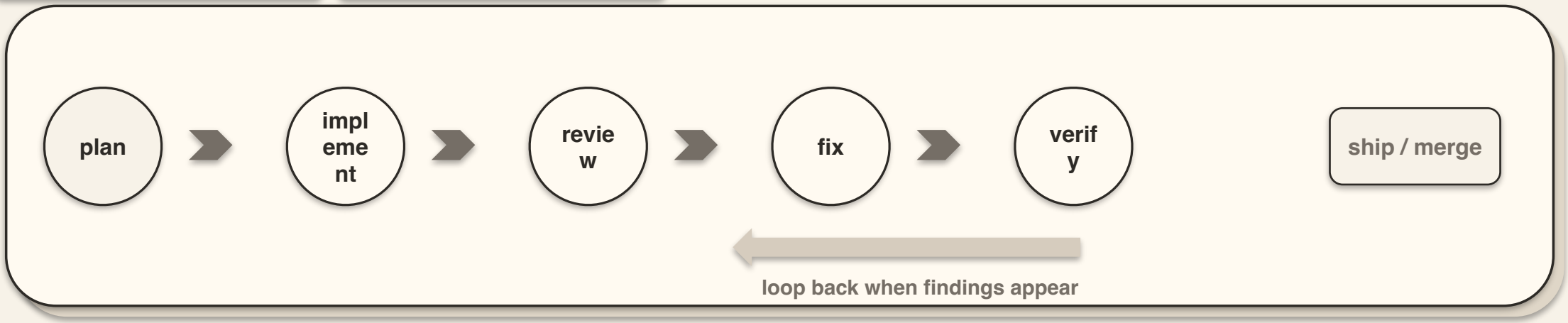
Implementation is an iteration loop, not a one-pass handoff from plan to done.

This is where hooks, guardrails, review, and verification become visible.

Workflow examples

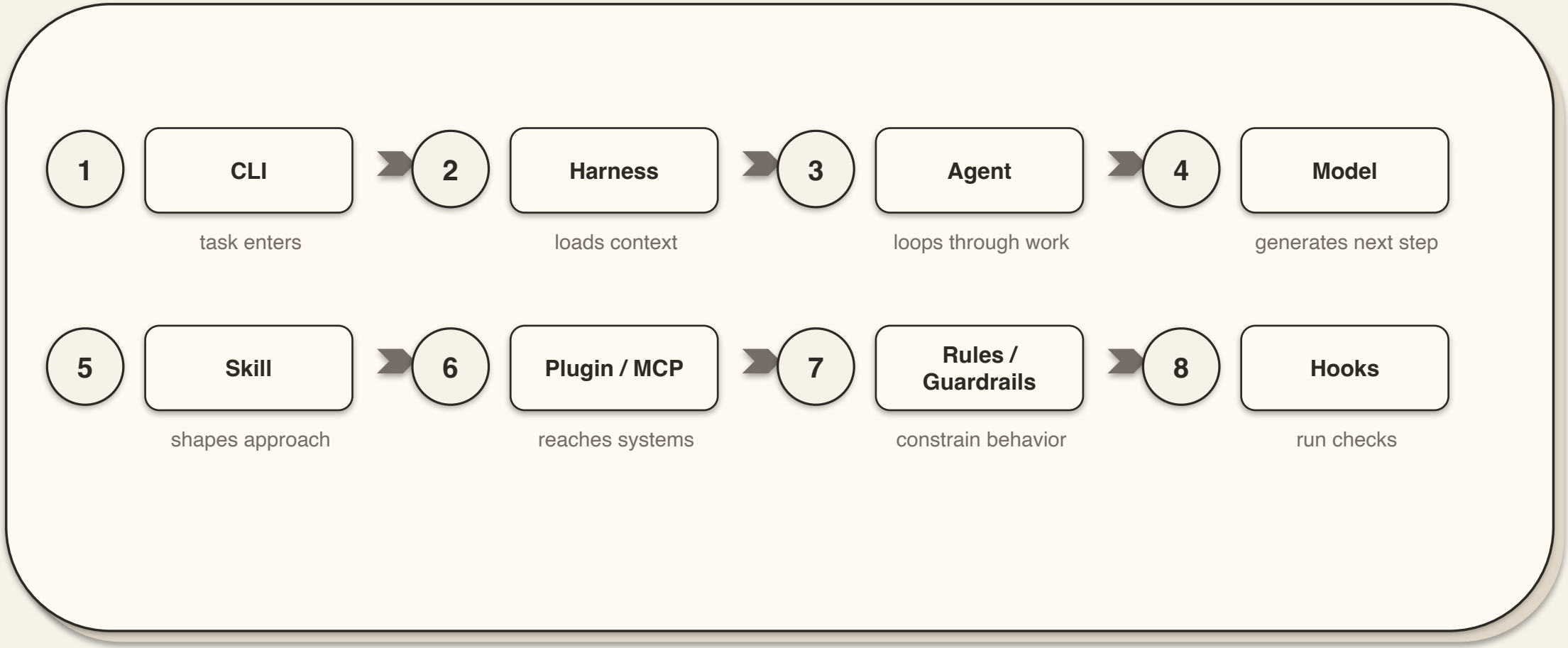
BMAD: quick-dev -> checkpoint

superpowers: execute -> review -> verify



END-TO-END EXAMPLE

One realistic flow: inspect a bug and propose a fix.



CHEAT SHEET

Screenshot this slide. It is the whole vocabulary in one place.

model	generates the next output
harness	runtime around the model
agent	goal-seeking execution loop
request	the current ask
instructions	standing guidance around the ask
rules	hard boundaries
skill	reusable workflow logic

plugin	added capability
MCP	common connection standard
CLI	command surface
token	small unit of text
context window	active information capacity
hook	trigger point in the process
guardrails	layered safety checks